

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 8

по курсу «Алгоритмы компьютерной графики»

Студент группы ИУ9-42Б Лавров Р. Д.

Преподаватель Цалкович П. А.

Москва 2026

1 Задача

Переписать на шейдерах одну из лабораторных: 2, 3 или 6.

Я выбрал лабораторную работу 6.

2 Основная теория

В классическом фиксированном конвейере OpenGL все этапы обработки вершин и фрагментов выполнялись по заданным правилам, а программист лишь настраивал параметры через функции вида $\mathrm{glLightfv}$, $\mathrm{glTexCoord2f}$ и т.п. С появлением программируемого конвейера (OpenGL 3.2+) разработчик получает прямой контроль над преобразованиями вершин, освещением и текстурированием с помощью **шейдеров** – небольших программ, исполняемых непосредственно на GPU. Шейдеры пишутся на языке GLSL (OpenGL Shading Language) и загружаются в графический конвейер в виде шейдерной программы.

В рамках лабораторной работы используются два основных типа шейдеров:

- **Вершинный шейдер** (vertex shader) – вызывается для каждой вершины и отвечает за преобразование координат из локального пространства модели в пространство отсечения, а также за подготовку интерполируемых данных (нормалей, текстурных координат) для фрагментного шейдера.

- **Фрагментный шейдер** (fragment shader) – вызывается для каждого потенциально видимого фрагмента (пикселя) и рассчитывает его итоговый цвет с учётом освещения, текстуры и других факторов.

Все шейдеры в проекте написаны на версии $\#\mathrm{version\ 330\ core}$, что соответствует профилю Core Profile OpenGL 3.3.

Для передачи геометрии в шейдеры вместо устаревших функций $\mathrm{glBegin/glEnd}$ используются буферные объекты:

- **Vertex Buffer Object (VBO)** – хранит атрибуты вершин (позицию, нормаль, текстурные координаты) в непрерывном массиве памяти GPU.

- **Element Buffer Object (EBO)** – содержит индексы, задающие порядок сборки треугольников из вершин VBO.

- **Vertex Array Object (VAO)** – контейнер, сохраняющий связи между VBO, EBO и входными атрибутами шейдера. Однократно настроенный VAO позволяет в цикле рисования выполнять только glBindVertexArray и glDrawElements, не повторяя конфигурацию.

Сфера строится в функции build_sphere путём разбиения на сегменты по широте и долготе. Для каждой вершины формируется кортеж из восьми чисел: трёх координат позиции, трёх компонент нормали и двух текстурных координат. Индексы объединяют вершины в треугольники против часовой стрелки, чтобы обеспечить правильную ориентацию граней.

Uniform-переменные

Параметры, единые для всех вершин или фрагментов в пределах одного вызова отрисовки, передаются через **uniform-переменные**. В работе через uniform задаются:

- матрицы модели, вида и проекции;
- характеристики источника света (позиция, направление, интенсивности ambient/diffuse/specular, угол отсечки и показатель затухания прожектора);
- свойства материала (ambient, diffuse, specular, блеск);
- флаг включения текстуры и сэмплер текстурного блока.

Значения uniform присваиваются функцией glUniform* после активации шейдерной программы.

Модель освещения

Используется классическая модель Фонга с прожекторным источником. Во фрагментном шейдере для каждой точки вычисляются:

- **ambient** – фоновая подсветка, складывающаяся из глобального ambient (параметр globalAmbient) и ambient-составляющей источника, умноженной на фактор прожектора;

- **diffuse** – рассеянное освещение по закону Ламберта ($\max(\mathrm{dot}(\mathrm{N},\ \mathrm{L}),\ 0)$), зависящее только от направления на источник и нормали поверхности;
- **specular** – зеркальный блик, учитывающий отражённый луч и направление на наблюдателя. Размер блика регулируется степенью $\mathrm{materialShininess}$.

Фактор прожектора $\mathrm{spotFactor}$ вычисляется как степень косинуса угла между направлением падающего света и осью конуса. Если фрагмент находится вне конуса (косинус меньше $\mathrm{spotCutoff}$), то фактор принимается равным 0.

Вектор направления на наблюдателя определён как $\mathrm{normalize}(\text{-}\mathrm{fragPos})$ – камера расположена в начале мировых координат (видовая матрица единичная). Нормаль переводится в мировое пространство умножением на матрицу $\mathrm{transpose}(\mathrm{inverse}(\mathrm{model}))$, что гарантирует корректность при неравномерном масштабировании.

Текстурирование

Текстура загружается из файла и накладывается на сферу с помощью текстурных координат, интерполированных из вершинного шейдера. Во фрагментном шейдере цвет текстуры выбирается встроенной функцией $\mathrm{texture}(\mathrm{texSampler},\ \mathrm{Texcoord})$ и умножается на вычисленный цвет освещения. Если текстурный режим отключён (клавиша T), используется только освещение с белым диффузным цветом материала.

Геометрические преобразования

Модельная матрица формируется перемножением справа налево матриц поворота вокруг осей Y и X, масштабирования и переноса, что соответствует последовательности преобразований фиксированного конвейера:

$$M = T(\mathbf{p}) \cdot S(s) \cdot R_x(\alpha) \cdot R_y(\beta).$$

Видовая матрица взята единичной – камера находится в центре координат. Проекционная матрица задана явно в виде трёхточечной перспективной проекции, аналогичной той, что использовалась в исходной лабораторной работе №6.

Таким образом, программируемый конвейер позволяет гибко заменить фиксированный функционал, сохраняя при этом математическую идентичность результата.

3 Результаты

```
import glfw
from PIL import Image
from OpenGL.GL import *
import numpy as np
import math
import time
import ctypes

angle_x, angle_y = 0.0, 0.0
wireframe = False
scale  = 0.8
texture_enabled = True

pos = np.array([-0.3,  0.0,  0.0], dtype=np.float32)
vel  = np.array([0.5,  0.4,  0.3], dtype=np.float32)
BOUND = 1.1
last_time = time.time()

def three_point_projection():
    return [0.87,  -0.09,  0.98,  0.49,
            0.0,    0.98,  0.35,  0.17,
            0.5,    0.15,-1.7,   -0.85,
            0.0,    0.0,   1.0,   2.0]

vertex_shader_source = """
#version 330 core
in vec3 position;
in vec3 vertexNormal;
in vec2 texcoord;

uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

out vec3 fragPos;
out vec3 fragNormal;
out vec2 Texcoord;

```

```
void main() {
    vec4 worldPos = model * vec4(position,1.0);
    fragPos = worldPos.xyz;
    fragNormal = mat3(transpose(inverse(model))) * vertexNormal;

    Texcoord = texcoord;
    gl_Position = projection * view * model * vec4(position, 1.0);
}
"""

fragment_shader_source = """
#version 330 core
in vec3 fragPos;
in vec3 fragNormal;
in vec2 Texcoord;

uniform sampler2D texSampler;
uniform bool textureEnabled;

uniform vec3 lightPos;
uniform vec3 lightDir;
uniform vec3 lightAmbient;
uniform vec3 lightDiffuse;
uniform vec3 lightSpecular;
uniform float spotCutoff;
uniform float spotExponent;

uniform vec3 materialAmbient;
uniform vec3 materialDiffuse;
uniform vec3 materialSpecular;
uniform float materialShininess;
uniform vec3 globalAmbient;

out vec4 outColor;

void main() {
    vec3 N = normalize(fragNormal);
    vec3 L = normalize(lightPos - fragPos);
    vec3 V = normalize(-fragPos);
    vec3 R = reflect(-L, N);

    vec3 spotDir = normalize(lightDir);
    float cosTheta = dot(-L, spotDir);
    float spotFactor = cosTheta < spotCutoff ? 0.0 : pow(cosTheta, spotExponent);

```

```
    vec3 ambient = globalAmbient * materialAmbient + lightAmbient * materialAmbient *
    spotFactor;
    float diff = max(dot(N, L), 0.0);
    vec3 diffuse = diff * materialDiffuse * lightDiffuse * spotFactor;
    float spec = pow(max(dot(R, V), 0.0), materialShininess);
    vec3 specular = spec * materialSpecular * lightSpecular * spotFactor;

    vec3 lightingColor = ambient + diffuse + specular;
    vec4 texColor = texture(texSampler, Texcoord);
    vec3 finalColor = textureEnabled ? texColor.rgb * lightingColor : lightingColor;
    outColor = vec4(finalColor, 1.0);
}
"""

def build_sphere(radius=1.0, segments=10, sides=20):
    vertices = []
    indices = []
    for i in range(segments):
        theta1 = -math.pi/2 + i/segments * math.pi
        theta2 = -math.pi/2 + (i+1)/segments * math.pi
        y1, y2 = radius*math.sin(theta1), radius*math.sin(theta2)
        r1, r2 = radius*math.cos(theta1), radius*math.cos(theta2)
        for j in range(sides+1):
            phi = j * 2*math.pi / sides
            x1, z1 = r1 * math.cos(phi), r1 * math.sin(phi)
            x2, z2 = r2 * math.cos(phi), r2 * math.sin(phi)
            nx1, ny1, nz1 = x1/radius, y1/radius, z1/radius
            nx2, ny2, nz2 = x2/radius, y2/radius, z2/radius
            u, v1 = phi/(2*math.pi), (y1/radius+1)/2
            v2 = (y2/radius+1)/2
            vertices.extend([x1,y1,z1, nx1,ny1,nz1, u,v1])
            vertices.extend([x2,y2,z2, nx2,ny2,nz2, u,v2])
        base = i*2*(sides+1)
        for j in range(sides):
            i0, i1, i2, i3 = base+2*j, base+2*j+1, base+2*j+2, base+2*j+3
            indices.extend([i0,i2,i1, i1,i2,i3])
    return np.array(vertices, dtype=np.float32), np.array(indices, dtype=np.uint32)

def create_texture(path='3.jpg'):
    img = Image.open(path).convert('RGB').resize((128,128))
    w, h = img.size
    tex = glGenTextures(1)
    glBindTexture(GL_TEXTURE_2D, tex)
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, w, h, 0, GL_RGB,
    GL_UNSIGNED_BYTE, img.tobytes())
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR)
```

```
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT)
    return tex

def translate_matrix(v):
    m = np.eye(4, dtype=np.float32)
    m[:3,3] = v
    return m

def scale_matrix(s):
    return np.diag([s,s,s,1]).astype(np.float32)

def rotate_x_matrix(deg):
    a = math.radians(deg)
    c, s = math.cos(a), math.sin(a)
    return np.array([[1,0,0,0],[0, c,-s,0],[0, s,c,0],[0,0,0,1]], dtype=np.float32)

def rotate_y_matrix(deg):
    a = math.radians(deg)
    c, s = math.cos(a), math.sin(a)
    return np.array([[ c,0,s,0],[0,1,0,0],[-s,0,c,0],[0,0,0,1]], dtype=np.float32)

def compile_shader(src, stype, name):
    sh = glCreateShader(stype)
    glShaderSource(sh, src)
    glCompileShader(sh)
    return sh

def make_program(vsrc, fsrc):
    vertex = compile_shader(vsrc, GL_VERTEX_SHADER, "vertex")
    fragment = compile_shader(fsrc, GL_FRAGMENT_SHADER, "fragment")
    prog = glCreateProgram()
    glAttachShader(prog, vertex)
    glAttachShader(prog, fragment)
    glLinkProgram(prog)

    glDetachShader(prog, vertex);
    glDetachShader(prog, fragment)
    glDeleteShader(vertex);
    glDeleteShader(fragment)
    glValidateProgram(prog)
    return prog

def key_callback(window, key, scancode, action, mods):
    global angle_x, angle_y, wireframe, scale, texture_enabled
```

```
    if action in (glfw.PRESS, glfw.REPEAT):
        if key == glfw.KEY_RIGHT:    angle_y -= 3
        elif key == glfw.KEY_LEFT:   angle_y += 3
        elif key == glfw.KEY_UP:     angle_x -= 3
        elif key == glfw.KEY_DOWN:   angle_x += 3
        elif key in (glfw.KEY_KP_ADD, glfw.KEY_EQUAL): scale += 0.1
        elif key in (glfw.KEY_KP_SUBTRACT, glfw.KEY_MINUS): scale = max(0.1, scale
    -0.1)
        elif key == glfw.KEY_M: wireframe = not wireframe
        elif key == glfw.KEY_T: texture_enabled = not texture_enabled

def display(shader_program, vao, num_indices, texture):
    global angle_x, angle_y, scale, pos, vel, last_time, texture_enabled

    current_time = time.time()
    dt = current_time - last_time
    last_time = current_time
    if dt > 0.1: dt = 0.016

    for i in range(3):
        pos[i] += vel[i] * dt
        if pos[i] > BOUND:
            pos[i] = BOUND
            vel[i] = -vel[i]
        elif pos[i] < -BOUND:
            pos[i] = -BOUND
            vel[i] = -vel[i]

    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

    model = translate_matrix(pos) @ scale_matrix(scale) @ rotate_x_matrix(angle_x) @
    rotate_y_matrix(angle_y)
    view = np.eye(4, dtype=np.float32)
    projection = three_point_projection()

    glUseProgram(shader_program)

    glUniformMatrix4fv(glGetUniformLocation(shader_program, 'model'), 1, GL_TRUE, model)
    glUniformMatrix4fv(glGetUniformLocation(shader_program, 'view'), 1, GL_TRUE, view)
    glUniformMatrix4fv(glGetUniformLocation(shader_program, 'projection'), 1, GL_FALSE,
    projection)

    glUniform3fv(glGetUniformLocation(shader_program, 'lightPos'), 1, [0, 3, 0])
    glUniform3fv(glGetUniformLocation(shader_program, 'lightDir'), 1, [0, -1, 0])
    glUniform3fv(glGetUniformLocation(shader_program, 'lightAmbient'), 1, [0.3, 0.3, 0.3])
    glUniform3fv(glGetUniformLocation(shader_program, 'lightDiffuse'), 1, [3, 3, 3])
```

```
217     glUniform3fv(glGetUniformLocation(shader_program, 'lightSpecular'), 1,[3, 3, 3])
218     glUniform1f(glGetUniformLocation(shader_program, 'spotCutoff'), math.cos(math.radians
        (70)))
219     glUniform1f(glGetUniformLocation(shader_program, 'spotExponent'), 70)
220
221     glUniform3fv(glGetUniformLocation(shader_program, 'materialAmbient'), 1, [1, 1, 1])
222     glUniform3fv(glGetUniformLocation(shader_program, 'materialDiffuse'), 1, [1, 1, 1])
223     glUniform3fv(glGetUniformLocation(shader_program, 'materialSpecular'), 1, [0.8, 0.8, 0.8])
224     glUniform1f(glGetUniformLocation(shader_program, 'materialShininess'), 64)
225     glUniform3fv(glGetUniformLocation(shader_program, 'globalAmbient'), 1, [0.3, 0.3, 0.3])
226     glUniform1i(glGetUniformLocation(shader_program, 'textureEnabled'), 1 if texture_enabled
        else 0)
227     glUniform1i(glGetUniformLocation(shader_program, 'texSampler'), 0)
228
229     if texture_enabled:
230         glActiveTexture(GL_TEXTURE0)
231         glBindTexture(GL_TEXTURE_2D, texture)
232     else:
233         glBindTexture(GL_TEXTURE_2D, 0)
234
235     glPolygonMode(GL_FRONT_AND_BACK, GL_LINE if wireframe else GL_FILL)
236     glBindVertexArray(vao)
237     glDrawElements(GL_TRIANGLES, num_indices, GL_UNSIGNED_INT, None)
238
239 def make_program(vsrc, fsrc):
240     vertex = compile_shader(vsrc, GL_VERTEX_SHADER, "vertex")
241     fragment = compile_shader(fsrc, GL_FRAGMENT_SHADER, "fragment")
242     prog = glCreateProgram()
243     glAttachShader(prog, vertex)
244     glAttachShader(prog, fragment)
245     glLinkProgram(prog)
246     return prog
247
248 def main():
249     global last_time
250     if not glfw.init():
251         return
252
253     glfw.window_hint(glfw.CONTEXT_VERSION_MAJOR, 3)
254     glfw.window_hint(glfw.CONTEXT_VERSION_MINOR, 3)
255     glfw.window_hint(glfw.OPENGL_PROFILE, glfw.OPENGL_CORE_PROFILE)
256     glfw.window_hint(glfw.OPENGL_FORWARD_COMPAT, GL_TRUE)
257     glfw.window_hint(glfw.COCOA_RETINA_FRAMEBUFFER, GL_FALSE)
258
259     window = glfw.create_window(800, 800, "lab8", None, None)
260     if not window:
```

```
261         glfw.terminate()
262         return
263
264     glfw.make_context_current(window)
265     glfw.set_key_callback(window, key_callback)
266
267     glEnable(GL_DEPTH_TEST)
268     glClearColor(0.15, 0.15, 0.15, 1.0)
269
270     temp_vao = glGenVertexArrays(1)
271     glBindVertexArray(temp_vao)
272
273     shader_program = make_program(vertex_shader_source, fragment_shader_source)
274
275     glBindVertexArray(0)
276     glDeleteVertexArrays(1, [temp_vao])
277
278     sphere_vertices, sphere_indices = build_sphere()
279
280     vao = glGenVertexArrays(1)
281     vbo, ebo = glGenBuffers(2)
282
283     glBindVertexArray(vao)
284     glBindBuffer(GL_ARRAY_BUFFER, vbo)
285     glBufferData(GL_ARRAY_BUFFER, sphere_vertices.nbytes, sphere_vertices,
        GL_STATIC_DRAW)
286     glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, ebo)
287     glBufferData(GL_ELEMENT_ARRAY_BUFFER, sphere_indices.nbytes, sphere_indices,
        GL_STATIC_DRAW)
288
289     stride = 8 * np.dtype(np.float32).itemsize
290
291     pos_loc = glGetAttribLocation(shader_program, "position")
292     glEnableVertexAttribArray(pos_loc)
293     glVertexAttribPointer(pos_loc, 3, GL_FLOAT, GL_FALSE, stride, ctypes.c_void_p(0))
294
295     norm_loc  = glGetAttribLocation(shader_program, "vertexNormal")
296     glEnableVertexAttribArray(norm_loc)
297     glVertexAttribPointer(norm_loc, 3, GL_FLOAT, GL_FALSE, stride, ctypes.c_void_p(12))
298
299     texcoord_loc = glGetAttribLocation(shader_program, "texcoord")
300     glEnableVertexAttribArray(texcoord_loc)
301     glVertexAttribPointer(texcoord_loc, 2, GL_FLOAT, GL_FALSE, stride, ctypes.c_void_p
        (24))
302
303     texture = create_texture('3.jpg')
```

```
    last_time = time.time()

    while not glfw.window_should_close(window):
        display(shader_program, vao, len(sphere_indices), texture)
        glfw.swap_buffers(window)
        glfw.poll_events()

    glDeleteVertexArrays(1, [vao])
    glDeleteBuffers(1, [vbo])
    glDeleteBuffers(1, [ebo])
    glDeleteProgram(shader_program)
    glDeleteTextures([texture])

    glfw.destroy_window(window)
    glfw.terminate()

if __name__ == "__main__":
    main()
```

4 Вывод

Исходная лабораторная работа №6, построенная на фиксированном конвейере, была полностью переписана с использованием программируемых шейдеров в контексте OpenGL 3.3 Core Profile. Вершинный шейдер выполняет преобразования координат и передаёт интерполируемые нормали и текстурные координаты во фрагментный шейдер, который рассчитывает освещение по модели Фонга с прожектором и накладывает текстуру. Все функциональные возможности оригинала (вращение, масштаб, движение с отражением, каркасный режим, отключение текстуры) полностью сохранены.